

编译原理 第十一次作业

姓名：朱首赫

学号：2023K8009906029

1 练习 7.1.1

考虑 C 语言的函数 f 和 g ：按照图 7-7 的约定，不考虑编译器优化，讨论当 h 调用 k 而 k 即将返回时运行时栈的状态，其中 h 的参数 $a = 2$ 。只需要讨论返回值、参数、控制链和代码中体现的局部数据。指出

```
1 int k(int *, int);
2 int h(int a) {
3     int i = a * 3 + 1;
4     return k(&i, a);
5 }
6 int k(int *p, int q) {
7     int s = *p + q;
8     return s + 2;
9 }
```

1. 哪个函数在栈中为各个元素创建了所使用的空间？
2. 哪个函数写入了各个元素的值？参数、返回值和局部变量的值是什么？
3. 这些元素属于哪个活动记录？

基本计算与分析： 当以 $a = 2$ 调用函数 h 时，在 h 中计算局部变量 $i = a \times 3 + 1 = 2 \times 3 + 1 = 7$ 。随后 h 调用 $k(\&i, a)$ ，即传递给 k 的实参为 $p = \&i$ （指向变量 i 的地址）， $q = 2$ 。在函数 k 中，计算局部数据 $s = *p + q = 7 + 2 = 9$ 。 k 的最终返回值为 $s + 2 = 9 + 2 = 11$ 。

按照一般的编译原理运行时刻环境约定（标准调用序列：调用者负责为被调用者的参数、返回值和控制链分配空间并入栈；被调用者负责为自己的局部数据分配空间），当 h 调用 k 且 k 即将返回时，运行时栈中各元素的状态详见下表：

表 1: 当 k 即将返回时运行时栈的状态表

栈中元素	1. 创建空间的函数	2. 写入值的函数	参数、返回值和局部变量的值	3. 所属活动记录
h 的参数 a	调用 h 的外层函数	调用 h 的外层函数	2	h
h 的局部变量 i	h	h	7	h
k 的返回值	h	k	11	k
k 的参数 p	h	h	h 中局部变量 i 的内存地址	k
k 的参数 q	h	h	2	k
k 的控制链	h	h	指向 h 的活动记录（旧 <code>top_sp</code> 指针）	k
k 的局部变量 s	k	k	9	k

2 练习 7.1.2

考虑下面的 Fibonacci 函数：嵌套在 fib0 中的是 fib1，它假设 $n \geq 2$ 并计算第 n 个 Fibonacci 数；嵌套在 fib1 中的是 fib2，它假设 $n \geq 4$ 。请注意，fib1 和 fib2 都不需要检查基本情况。我们考虑从对 main 的调用开始，直到（对 fib0(1) 的）第一次调用即将返回的时段。

1. 请描述出当时的活动记录栈，并给出栈中的各个活动记录的访问链。

2. 假设我们使用 display 表来实现下图中的函数。请给出 fib0(1) 的第一次调用即将返回时的 display 表。同时指明那时在栈中的各个活动记录中保存的 display 表条目。

```

1 fun main() {
2   let
3     fun fib0(n) =
4       let
5         fun fib1(n) =
6           let
7             fun fib2(n) = fib1(n-1) + fib1(n-2)
8           in
9             if n >= 4 then fib2(n)
10            else fib0(n-1) + fib0(n-2)
11          in
12            if n >= 2 then fib1(n) else n
13          end
14        in
15          fib0(5)
16        end;
17  }

```

函数嵌套深度与调用链分析： 在静态源程序中，各函数的嵌套深度定义如下：

- main: 嵌套深度为 1
- fib0: 嵌套深度为 2（直接嵌套在 main 中）
- fib1: 嵌套深度为 3（直接嵌套在 fib0 中）
- fib2: 嵌套深度为 4（直接嵌套在 fib1 中）

当执行 fib0(5) 时，假设表达式按照自左向右的顺序求值，其函数调用链如下：

```

main() → fib0(5)  (5 ≥ 2, 调用fib1)
        → fib1(5)  (5 ≥ 4, 调用fib2)
        → fib2(5)  (计算fib1(4) + fib1(3), 先调用fib1(4))
        → fib1(4)  (4 ≥ 4, 调用fib2)
        → fib2(4)  (计算fib1(3) + fib1(2), 先调用fib1(3))
        → fib1(3)  (3 < 4, 进入 else 分支计算fib0(2) + fib0(1), 先调用fib0(2))
        → fib0(2)  (2 ≥ 2, 调用fib1)
        → fib1(2)  (2 < 4, 进入 else 分支计算fib0(1) + fib0(0), 先调用fib0(1))
        → fib0(1)  (1 < 2, 直接返回1)

```

因此，当对 fib0(1) 的第一次调用即将返回时，上述所有函数均处于活跃状态，且都在运行时栈中。

1. 活动记录栈及访问链（静态链）描述

每个活动记录（AR）的访问链（Access Chain / Static Link）必须指向在静态文本中直接包围它的、最近的那个外层函数的活动记录。下表展示了此时从栈顶（最新调用）到栈底（最早调用）的活跃记录状态及对应的静态链指向：

表 2: 当 fib0(1) 即将返回时的运行时栈与访问链状态

栈中位置	活动记录 (参数)	嵌套深度	访问链 (Static Link) 指向
栈顶 (Top)	fib0(1)	2	指向 main
	fib1(2)	3	指向 fib0(2)
	fib0(2)	2	指向 main
	fib1(3)	3	指向 fib0(5)
	fib2(4)	4	指向 fib1(4)
	fib1(4)	3	指向 fib0(5)
	fib2(5)	4	指向 fib1(5)
	fib1(5)	3	指向 fib0(5)
	fib0(5)	2	指向 main
栈底 (Bottom)	main()	1	无 (或指向 NULL)

2. Display 表状态及各个活动记录中保存的 Display 条目

Display 表用于加速非局部名字的访问，*Display*[*i*] 始终指向当前栈中**最新创建且仍活跃的**、嵌套深度为 *i* 的活动记录。当一个嵌套深度为 *i* 的新活动记录被创建时，它会首先将当前 *Display*[*i*] 的值保存到该活动记录的特性域中，然后将 *Display*[*i*] 更新为指向它自己。

此时的 Display 表状态

在 fib0(1) 处于栈顶时，各嵌套深度上最新的活跃记录如下：

- **Display**[1] → main()
- **Display**[2] → fib0(1) （替代了先前的 fib0(2) 和 fib0(5)）
- **Display**[3] → fib1(2) （替代了先前的 fib1(3)、fib1(4)、fib1(5)）
- **Display**[4] → fib2(4) （替代了先前的 fib2(5)）

各个活动记录中保存的 Display 条目

按照函数调用的时间先后顺序（从栈底到栈顶），各活动记录中保存的旧 Display 条目值如表 3 所示：

表 3: 各个活动记录中保存的 Display 条目（按调用历史排序）

创建顺序	活动记录 (AR)	自身深度	保存的旧 Display 条目及值
1	main()	1	无
2	fib0(5)	2	保存了旧的 <i>Display</i> [2] = NULL
3	fib1(5)	3	保存了旧的 <i>Display</i> [3] = NULL
4	fib2(5)	4	保存了旧的 <i>Display</i> [4] = NULL
5	fib1(4)	3	保存了旧的 <i>Display</i> [3] = fib1(5)
6	fib2(4)	4	保存了旧的 <i>Display</i> [4] = fib2(5)
7	fib1(3)	3	保存了旧的 <i>Display</i> [3] = fib1(4)
8	fib0(2)	2	保存了旧的 <i>Display</i> [2] = fib0(5)
9	fib1(2)	3	保存了旧的 <i>Display</i> [3] = fib1(3)
10	fib0(1)	2	保存了旧的 <i>Display</i> [2] = fib0(2)